day-1

Linear Regression :-

It's a Supervised Machine learning Algorithm that is used to predict a continuous value by finding the relation ship b/w input (x), output (y) feature.

Ex. ① predicting Salary based on the experience.

② predicting house price based on area.

Working Process of linear Regression Model :-

① Take input/output feature

② Initialize random weight and bias.

③ Now apply linear regression equation ($y = mx + b$)

$$y = m_1x_1 + m_2x_2 -- m_nx_n + b$$

④ Now we will predict the value ($\hat{y}$).

⑤ Now Compare predicted value and the actual values.

⑥ Now Calculate the loss function.

⑦ Now adjust weight and bias to reduce the error.

⑧ Repeat the prediction and error calculation multiple time.

github.com/krishkydv77

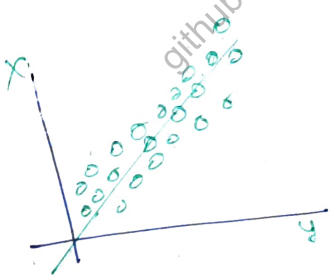
④ Now we will find the best fit line with Minimum error.

⑤ Now we will ~~find the~~ final trained Model for prediction on ~~testing~~ Unseen or new data

Metric :-

- MAE ⇒ Mean Absolute Error ⇒ b
- MSE ⇒ Mean Squared error
- RMSE ⇒ Root Mean Squared error
- R^2 - Score ⇒

Metric	Why we use	Want to use
① MAE	It given avg. error without heavily Mistake.	When all error should be treated equally.
② MSE	Squaring increasing the Impact of large error	When large error are very dangerous and Important.
③ RMSE	Want When you want error in the Same Unit as target Variable and also want Punish large error	Also easy to interpret and Sensitive to outliers.
④ R2-Score	It Show overall Model percentage.	When you want to how well the Model explain the data.



x↑ & y↑

Interview

① What is the main goal of linear Regression.

⇒ The main goal of linear Regression is to find the relationship between input data and output data, and predict continuous values based on that relationship.

→ It draws a best-fit straight line b/w input and output data to make predictions.

② What is different Simple linear and Multiple linear regression.

⇒ Both are main difference is the number of ~~used~~ independent variables used.

Simple linear ⇒ SL-Regression uses only one (Input Independent) Variable to predict the dependent variable.

Ex. predicting Salary based on experience.

Multiple linear Regression uses two or more independent variable to predict the dependent variable

Ex. predicting Salary based on experience, skills, and education.

Q: what is the best fit line in linear regression?

⇒ The best fit line in linear Regression is the line that show the relationship b/w input/output variable with minimum error.

⇒ It is the line that fits the data points most accurately and helps in making prediction.

Q: what is Slope (→ y=mx+c) and intercept Represent in L.R.?

⇒ → Slope represent how much the dependent variable changes when the independent variable increases by one unit

→ Intercept represent the value of the dependent variable when the independent variable is zero.

Q5. When should you use linear Regression and when should you not use linear Regression.

use linear Regression when there is a linear relationship b/w input and output variable and the target value is continuous, like salary prediction or sales prediction.

Do not use linear Regression when:-

→ data has no linear Relationship.

→ output is Categorical.

→ data ~~Infinite~~, contains many outliers and complex patterns.

Q6 what is difference b/w overfitting and underfitting.

→ overfitting happens when the Model learn training data too well, including noise and performs poorly on new data.

→ Underfitting happens when the Model cannot learn the pattern of the data properly and given poor performance on both training and testing data.

Q:7 what do you Regularization technique.

⇒ Regularization techniques are methods used to reduce overfitting in Machine learning Models.

They help improve Model performance by adding a penalty to large coefficient value and making the Model simpler.

github.com/krishkydv77

type of Regularization

① L1 Regularization (Lasso) → Can reduce some coefficients to zero and helps in feature selection.

② L2 Regularization (Ridge) → Reduce coefficients value but does not make them zero. Reduce overfitting, Correlated features

③ Elastic Net → Combination of both L1 and L2 regularization.

alph ⇒ control penalty strength

① Logistic Regression? why is it called regression even though it is used for classification?

Ans Logistic Regression is a supervised Machine learning algorithm used for classification problems, Mainly for ~~that~~ predicting Yes/No, True/false or 0/1 output.

It is called Regression because internally it use a Mathematical regression equation to calculate probability. But the final output is converted into classes using the Sigmoid function.

github.com/krishkydv77

② What is the different b/w linear and Logistic Regression?

Linear Regression	Logistic Regression
① used for prediction	used for classification.
② output is continuous value	output is probability
③ use straight line	use Sigmoid curve
④ output can be any number	output is b/w 0 and 1.

③ What is Sigmoid function and why is it used in Logistic Regression?

Ans The Sigmoid function converts any value into a probability b/w 0 and 1.

$$y = \frac{1}{1+e^{-x}}$$

Why used

① Logistic Regression given output from $(-\infty$ to $\infty)$

② Sigmoid convert it into probability

③ Helps in binary classification.

④ What happen when input is very large positive or very large Negative.

⇒ for Sigmoid function-

① Very large positive
→ Sigmoid output becomes close to 1.
→ Model predicts positive class.

② Very large negative
→ Sigmoid output becomes close to 0.
→ Model predicts negative class.

⑤ Does Logistic Regression require normal Distribution and feature scaling.

Normal Distribution

No, Logistic Regression does not require normal distribution.

Feature Scalling

Feature Scalling is not Mandatory, but recommended when:

- -> Feature have very different ranges.
- -> using gradient descent regularization.

⑥ What is Confusion Matrix?

Confusion Matrix is used to evaluate classification Model Performance.

TP (True positive) ⇒ Model Correctly predicted positive.

TN (True Negative) ⇒ Model correctly predicted Negative.

FP (False positive) ⇒ Model predicted positive but actually negative.

FN (false Negative) ⇒ Mode predicted negative but actually positive.

⑦ What is Precision, Recall and f1-score.

① Precision

out of all predicted positive, how many were actually positive

$$\text{precision} \Rightarrow \frac{TP}{TP+FP}$$

High precision means fewer false positive

② Recall

out of all actual positive, How Many were Correctly predicted.

$$\text{Recall} = \frac{TP}{TP+FN}$$

High recall means fewer false negative.

③ f1-Score.

Blance b/w precision and recall.

$$f1 = 2 \times \frac{\text{precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

used when dataset is imbalanced.

github.com/krishkydv77

⑧ What is class imbalance? How do you handle imbalanced datasets.

Class imbalanced

When on class has much more data than another class.

Problem → Model becomes biased toward Majority class.

Handling imbalanced dataset:-

① oversampling → Increase minority class samples.
Ex. SMOTE

② Undersampling → Reduce majority class samples.

③ Use different Evaluation Metrics.

⑨ When should Logistic Regression be avoided?

Avoid Logistic Regression when.

① Data is highly non-linear
→ Logistic Regression work best on linear relationship.

② Very complex dataset
↳ tree-based Model may work better.

③ Large number of irrelevant feature.
Too Many useless feature reduce performance.

④ Multi-class Complex problem
↳ Advance Models.

Note

Logistic Regression is a simple and powerful classification that predicts probability using the Sigmoid function. It works well for binary classification problems and is easy to interpret.

① What is decision tree?

Ans: A decision tree is a supervised ML model. It is use for classification and regression problems.

It works likes a tree structure where data is divied step by step based on condition.

Structure of decision tree ⇒

① Root Node ⇒ Starting point of the tree

② top node ⇒

③ this is starting node of the tree. It is represent it entire dataset and first feature selection happen here and the best split is

② Branches ⇒ Branches represent decision Rule or condition and branches connect on node to another node.

③ leaf Nodes ⇒ this is final output node and there will be no more splitting occurs it given find prediction.

How decision tree classifier works.

① take complete training dataset

② select the best feature for splitting data.

③ Calculate Impurity measure like Gini Indix, Entropy and Information gain.

④ choose feature best split

⑤ It will be create root node

⑥ now divied dataset into subset using condition.

⑦ repeat the same process recursively for each subset

⑧ Now continue splitting until pure class are formed.

⑨ now it will be create leaf node with find class labels.

⑩ during the prediction input travel through condition and.

Working process of decision tree Regression:-

① Take Complete training dataset
② Select best feature for splitting data
③ Now it will be Calculate error reduction use matrix like mean square error.
④ Now it will be choose split with Minimum error.
⑤ Now it will be create root nodes.
⑥ Now it will be split the dataset into Smaller subset
⑦ Now repeat recursive splitting process
⑧ Continue until stoping create point.
⑨ Now leaf node store the avg. traget value.
⑩ during the prediction input follower dicision rules and it gich leaf node and the output numerical value predicted.

① What is a Decision Tree?

A Decision Tree is a Machine learning algorithm used for classification and Regression. It makes decision by splitting data into Smaller groups based on feature values.

② why is it Called a Tree Model? What are Root Node, Internal Node, and leaf Node

⇒ It is Called a tree because its structure look like an upside-down tree.

Root Node → Starting point of the tree
Internal Node → Decision points where data is Split
Leaf Node → final prediction or output

③ How does a Decision Tree make decision.

⇒ A Decision Tree asks a series of question. It start from the root node and follow branches based on feature value until it reaches a leaf node.

github.com/krishkvdv77

④ How does the model decide the best feature for splitting?

⇒ The model checks all feature and chooses the one that creates the purest groups after splitting.

It calculates metrics such as:

→ Gini Impurity

→ Entropy / Information Gain

The feature the reduces impurity the most becomes the split.

⑤ What is Impurity in Decision Trees?

⇒ Impurity means how mixed the classes are in a node.

Confused

⑥ What are Gini Impurity and Entropy?

⇒ Both measure impurity in a node.

Gini Impurity

Measures how often a randomly selected sample would be incorrectly classified.

Gini = $1 - \sum p_i^2$

→ fast to compute

→ used by default in many Decision tree implementations

Entropy → measur impurity in data

Measures randomness or uncertainty.

Entropy = $-\sum p_i \log_2(p_i)$

→ Based on information theory

→ Used to calculate information Gain.

Web → measures probability of incorrect classification.

Both Gini and Entropy measure impurity. The model chooses the split that reduces impurity the most. Gini is faster, while Entropy measure information gain.

⑦ Why would you choose Decision Tree over Logistic Regression

choose Decision tree when:-

① Relationships are non-linear

② Data has many categorical feature.

③ Easy interpretation is required.

④ No feature scaling is needed.

⇒ I would choose a Decision Tree when the relationship between variables is complex and non-linear. Decision Trees are easy to understand and do not require feature scaling.

(3) When should Decision Tree be avoided?

⇒ Avoid Decision Trees when:-

→ Dataset is very small —

→ High accuracy and generalization are required.

→ Data is noisy —

→ overfitting is a major concern.

I would avoid a Single Decision Tree when overfitting is likely or when better generalization is needed. In such cases Random Forest or Gradient Boosting may perform better.

(4) How can you reduce overfitting in Decision Tree?

Methods

① Limit tree depth (max_depth)

② Increase Minimum Samples per leaf (min_samples_leaf)

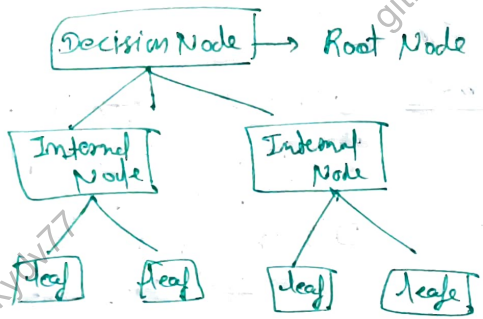
③ Increase minimum samples required to split (min_samples_split)

④ Pruning the tree

⑤ Use Random forest instead of a single tree.

⇒ overfitting can be reduced by limiting the depth of the tree, increasing minimum samples for splitting or leaf nodes, pruning unnecessary branches, and using ensemble methods like Random forest.

github.com/krishkydv77



github.com/krishkydv77

① Why does Random forest perform better than a Single Decision Tree?

⇒ A Single decision tree can easily overfit the training data. Random forest combines many decision tree and take the majority vote or average. This reduces overfitting and improves accuracy.

② what is n_estimators?

n_estimators is the number of decision trees in the Random forest?

③ What happen if n_estimators increases too much?

⇒ Model accuracy may improve slightly.—
→ Training times increases.
→ Memory usage increases.
→ After a certain point, performance becomes very small.

④ what is max_depth?

⇒ Max_depth controls the maximum depth of each decision tree.

Small depth → Simpler Model, less overfitting

Large depth → Complex Model, may ~~overfitting~~ overfit

⑤ What is Min_Samples_split?

⇒ It is the Minimum number of Samples required to Split a node into child node.

⑥ what is Min_Samples_leaf?

⇒ It is the Minimum number of Samples that must remain in a leaf node.

⑦ what is Max_features in Random forest? "sqrt"

⇒ Max_feature determines how many features are considered when finding the best split at each node.
→ Create diversity among trees.
→ Reduces correlation b/w trees.
→ Improve generalization.

github.com/krishkydv77

Why Random forest over Decision Tree?

⇒ Decision Trees can overfit the data. Random forest combines multiple trees and averages their predictions which reduces overfitting and improves accuracy.

① Your Decision Tree model gives 100% training accuracy but poor test accuracy. What will you do?

⇒ This is a sign of overfitting. The model has memorized the training data but cannot generalize to new data.

What I would do:

→ Reduce Max_depth
→ Increase Min_Samples_split / leaf.
→ Prune the tree.
→ Use Random forest instead of a Single Decision Tree.
→ Collect more training data if possible.

② your dataset has 500 features. Why can Random forest work well?

⇒ Random forest does not use all features at every split. It randomly selects a subset of features (max_features) for each split

Benefits

→ Handles high-dimensional data well
→ Reduces overfitting
→ Makes trees different from each other.
→ Can identify important features

Note:

Random forest works well with many features because it randomly selects a subset of features at each split reducing overfitting and improving performance.

③ Can Random forest handle Missing Values.

⇒ Standard Scikit-learn Random forest cannot directly handle Missing Values, so I would perform data imputation before training.

github.com/krishkydv77

Q) Can Random forest handle categorical data?

⇒ Yes, Random forest can work with categorical data. But in Scikit-learn the categorical usually need to be encoded before training.

Naive Bayes:-

Naive Bayes is a supervised ML algo. based on bayes theorm. It is called naive because it assume that all input feature are independent.

⇒ It is a probabilistic classification algo that predicts output classes based on probability Calculation using bayes theorm.

Core working idea of Naive bayes:

What is the probability that this input belong to a particular class so the class with highest probability becomes the final prediction.

formula of bayes theorm:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

$P(A|B)$ ⇒ Posterior Probability
- ↳ Probability of A given B.
- ↳ After seeing B, how likely is A.

$P(B|A)$ ⇒ likelihood (A होने पर B की probability)

↳ Probability of B when A is true

↳ If A is true, how likely is B.

$P(A)$ ⇒ prior probability

→ probability of A before seeing any evidence

$P(B)$ ⇒ evidence

⇒ Total probability of B occurring.

Ex.

Training data

Email words	class
offer, win, money	Spam
win, lottery, offer	Spam
Meeting, project, Tomorrow	Not Spam
project, Discussion, Meeting	Not Spam
win, Money, offer	Spam

Step① Calculate prior probability.

Spam Email

$$P(spam) \Rightarrow \frac{3}{5} = 0.6$$

Not Spam

$$P(Not\ spam) = \frac{2}{5} = 0.4$$

Step② Calculate Conditional probability.

Total word = 9

probability of "win" given Spam =

$$P(win/spam) = \frac{3}{9} = 0.33$$

$$P(offer/Spam) = \frac{2}{9} = 0.33$$

Step③ Apply Navie Bayes.

$$P(spam) \times P(win/spam) \times P(offer/spam) = 0.6 \times 0.33 \times 0.33$$

$$\Rightarrow 0.065$$

for Not Spam

win and offer are not present in Not Spam email.

So probability becomes very small

So probability = 0.

Type of Naive Bayes :-

① Gaussian Naive Bayes
- ↳ Continuous Numerical Data.
- ↳ Data follow Normal distribution.

② Multinomial Naive Bayes
- ↳ Used → Text classification

Work on
- ↳ word Counts
- ↳ word frequency.

③ Bernoulli Naive Bayes :-
- ↳ Binary features.

Values
- ↳ 0 or 1
- ↳ True / false
- ↳ Yes / no

Disadvantages :-
① Assume all features are independent
② Less accurate when feature are highly Correlated
③ zero frequency problem can occur
④ Cannot learn complex relationships between features.

⑧

github.com/krishkydv77

① What is Naive Bayes? Why is it called Navie Bayes?

Ans Naive Bayes is a supervised ML classification algorithm based on Bayes theorm. It predicts the probability of each class and chooses the class with the highest probability.

It is called "Navies" because it assumes that all input feature are independent of each other, which is often not in real-world data.

② How does Naive Bayes predict output classes?

Naive Bayes calculates the probability of each class using Bayes theorem.

It computes:

P(class/feature)

for every class and selects the class with the highest probability as the final prediction.

github.com/krishkydv77

③ Does Naive Bayes work well with dependent features? why?

⇒ No, Naive Bayes does not work well when features are highly dependent or correlated.

Because its main assumption is that all features are independent. When this assumption is violated, prediction accuracy may decrease.

④ What are the type of Naive Bayes?

① Gaussian Naive Bayes
↳ used for continuous numerical data
↳ Assumes data follows a normal distribution.

② Multinomial Naive Bayes.
↳ used for text classification.
↳ works with word counts and frequencies

③ Bernoulli Naive Bayes.
↳ used for binary feature
↳ feature have values like 0/1 or True/False.

⑤ if features are highly correlated, will you use Naive Bayes

⇒ Generally No.
Since Naive Bayes assumes feature independence, highly correlated feature can reduce its performance.
In such cases, I would prefer algorithms like Decision Tree, Random forest.

⑥ if the dataset is small, would you prefer Naive Bayes.

⇒ Yes, Naive Bayes works very well on small datasets because it requires less training data and learns quickly.

⑦ if training speed is important, would you use Naive Bayes?

Yes, Naive Bayes is one of the fastest Machine learning algorithm for both training and prediction, making it suitable when speed is important.

github.com/krishkydv77

⑧ If data contains many categorical or text features, would you use Naive Bayes?

Yes, Naive Bayes performs very well on text and categorical data. Multinomial Naive Bayes is commonly used for Spam detection, Sentiment analysis, and document classification.

⑨ What are the limitations of Naive Bayes?

① Assumes all feature are independent
② Performs poorly when feature are highly correlated
③ Suffers from the zero frequency problem.
④ Cannot capture complex relationship between feature
⑤ Sometime less accurate than advanced algorithm like Random forest.

⑩ Is feature scaling required in Naive Bayes?

No, feature scaling is generally not required in Naive Bayes because it is probability-based and does not depend on distance calculations. Algorithm like KNN, K-means and SVM require scaling but Naive Bayes usually does not.

Algorithm	Use when	Don't use when	Data type	Main Advantage	Main Limitation
① Linear Regression	Target = Continuous, linear relationship	Relationship is highly complex or non-linear	Numerical output	Simple, fast, easy to interpret	Cannot capture complex patterns
② Logistic Regression	classification problem with 2 classes (0/1)	complex non-linear relationship exist	categorical output	fast, interpretable	Struggles with complex boundaries
③ Decision Tree	Need easy interpretation and business explanation	Large tree can overfit	classification or Regression	easy to understand and visualize	High overfitting risk
④ Random forest	Need high accuracy and robust model	Very small datasets or when explainability is important	" "	Reduce overfitting, High accuracy	-less interpretable, slower
⑤ Naive Bayes	Text classification, Spam detection, Small dataset	feature are highly Correlated	Mainly classification	Very fast, work well on text data	① Assume feature independence

Metric	When we use it	Why we use it
MAE	When all prediction errors should be treated equally	Because it gives the avg. error directly and it easy to understand.
MSE	When large errors are very dangerous and should be penalized heavily	Because squaring increases ~~interpret and also punishes large errors~~ the impact of large mistakes
RMSE	When we want error in the same unit as output variable	Because it is easy to ~~interpret~~ compare and also punishes large errors
R^2 Score	When we want to know how well do Model fits the data	Because it shows how much variance is explained by the Model

① MAE
When? → Use when outliers are not very important
→ Use when we want simple avg. error
$\frac{1}{n}\sum|y_i - \hat{y}_i|$
→ easy to explain to business people
→ Given real error value

② MSE
$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$
→ Large mistake are unacceptable
→ Common in ML training
→ squaring increases punishments for big error
→ Helps optimization algorithms work better

③ RMSE
$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$
→ We need interpretable error
→ most common regression metric in industry
→ output remains in original unit
→ Better understand of prediction quality.

④ R^2 Score
$1 - \frac{SS_{res}}{SS_{tot}}$
→ Measure overall Model performance
→ Compare multiple regression models.
→ Show how much data variation Model explains
→ Higher R^2 means better fitting

SVM

SVM is particularly effective in dealing with complex and high-dimensional datasets. The fundamental principle of SVM is to find an optimal hyperplane that maximally separates different classes in the input space.

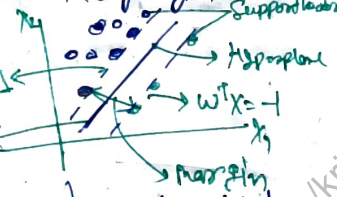
How SVM works:-

SVM requires ~~data~~ labeled training data consisting of input features and corresponding class labels. Each data point is represented as a feature vector where each feature describes a particular characteristic of the data point. The data points should be preprocessed and scaled to ensure the features are on similar scales, typically between 0 and 1.

What is hyperplane and margin?

SVM aims to find a hyperplane that best separates the different classes in the feature space. In a binary classification problem, the hyperplane is a line in a 2D space or a hyperplane in a high-dimensional space.

SVM seeks maximum margin, which is the distance between the hyperplane and the nearest data points from each class. The points on the margin are know as support vectors as they play a crucial role in defining the decision boundary.



What is linear SVM?

The linear SVM finds a linear hyperplane that separates the classes. The goal is to find the hyperplane that maximizes the margin while minimizing the misclassification of training data.

Non linear SVM?

In cases where the data is not linearly separable, SVM uses a technique called the Kernel trick. The Kernel Trick maps the original input space into a higher-dimensional feature space where the data points can be linearly separable.

~~train~~
training =>

github.com/krishkydv77

① What are Support Vectors? Why are they Important?

⇒ Support Vectors are the nearest data point to the decision boundary. These points support the hyperplane.

Support Vectors Important:-

Because they determine the position of the boundary, without them, SVM boundary changes.

⇒ SVM mainly depends on support vectors, not all points.

② What is a Hyperplane?

Hyperplane ⇒ Separating boundary

Depending on dimensions:-

In 2D → line

In 3D → plane

In higher dimensions ⇒ hyperplane.

③ Why does SVM not use all data point to create the boundary?

⇒ Because only the nearest points matter. Far-away points do not affect the ~~bound~~ boundary much.

SVM only focuses on:- [Support Vectors — critical boundary points

Advantage — faster prediction
Better generalization
Less affected by unnecessary data.

④ If Support Vectors are removed what happen?
if Support Vectors are removed then Boundary change Completely. Model may create different hyperplane.
Because support Vector define the margin.
But Removing far way points usually change nothing.

⑤ Why does SVM Maximize the margin.
Because large Margin means:-
→ Better Separation
→ Lower overfitting
→ Better Performance on unseen data.

⑥ What is Kernel Trick in SVM? Why do We need Kernels?
⇒ Sometimes data is:
Not linearly separable means A Straight line cannot separate them. So SVM uses.
Kernel Trick.
Kernel Convert data into higher dimensions where Separation becomes easier.
2D → 3D
Then a plane can separate classes.
Why do we need Kernels?
Because real word data is often Complex.
Without Kernels → SVM work only on Simple linear data
With Kernels → SVM handles non-linear data.
Common Kernels types:-
Linear Kernel → used for linear data
Polynomial Kernel → Create curved boundaries.
RBF Kernel → Handles highly Complex data
Sigmoid Kernel → look like neural network behavior.

⑦ What is Soft Margin and Hard Margin?

Hard Margin SVM → No Misclassification allowed
Data must be Perfectly separable
↳ Problem
↳ Very Sensitive to outliers

Soft Margin SVM → Allow some Mistakes.
↳ Focuses on better generalization.
Used in real-world datasets because:
Real data Contains noise.

⑧ Why does SVM perform well with small datasets?
Because:
→ SVM focuses only on important points
→ Works well in high-dimensional data
→ Good mathematical optimization
Especially useful when:
→ Features are many
→ Record are fewer

⑨ Why is SVM not preferred for Millions of records?
→ Because training becomes:
→ Very slow
→ Memory expensive.

Reason: SVM optimization is Computationally heavy.
problem with huge datasets:
High training time
Large Memory Usage
Kernel Matrix becomes massive

So for Very large data: Random forest
X Gboost
Deep learning.
are often preferred.

⑩ Why does SVM use Support Vectors Instead of Centroids?
→ Centroid Method → use center points
But SVM focuses on boundary points.
Why?
Because classification depend more on:
Where classes touch each other not on the center.

KNN

K-Nearest Neighbore (KNN) :-

KNN is a supervised ML algorithm used for both classification and regression.
It predicts the class/value of a new data point based on the KNN from the training dataset

Working step of KNN :-

① choose the value of k
② Calculate the distance between the test point and all training points
③ Select the k nearest neighbors.
④ for :-
→ classification = use majority voting
→ Regression = Take the average of k nearest values.
⑤ Assign the final prediction.

Q Why is KNN called a lazy learner?
⇒ ~~Because predictions are~~ KNN performs no actual training. It simply stores the training data and performs all computation during prediction time.

Therefore, it it called as lazy learning Algorithm.

Why is KNN called an Instance Based learning Algorithm?

⇒ Because predictions are made directly using the stored training instances rather than building a mathematical model.

Therefor, KNN is known as an Instance-Based learning Algorithm.

Why is KNN a Non-parametric Algorithm?

⇒ KNN does not assume any fixed probability distribution or function form of the data.

Therefore, it is called a Non-Parametric Algorithm.

Effect of K-Value :-

Small K (K=1)
- ↳ Sensitive to noise
- ↳ High variance
- ↳ Can cause overfitting.

large K
- ↳ High bias
- ↳ Can cause underfitting.
- ↳ Smoother decision boundary

Rule for selecting K

A common rule is:

$$K = \sqrt{N}$$

Where:

→ N = Number of training samples

⇒ Usually choose an odd value of K for binary classification to avoid ties.

Distance Metrics used in KNN

1. Euclidean distance
2. ~~Manth~~ Manhattan distance
3. Minkowski distance

Euclidean distance formula:

$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

Advantage of KNN :-

1. No training phase required.
2. Easy to understand and implement
3. Effective on small datasets
4. Naturally handles multiclass classification.
5. Can be used for both classification and regression.

Disadvantages of KNN :-

1. Computationally expensive during prediction.
2. Memory intensive because it stores all training data.
3. Sensitive to noisy data and outliers.
4. Performance decreases in high-dimensional data
5. Requires feature scaling.

Why is feature scaling important in KNN?

KNN is distance-based, features with larger values can dominates the distance calculation.

Therefore we use :-

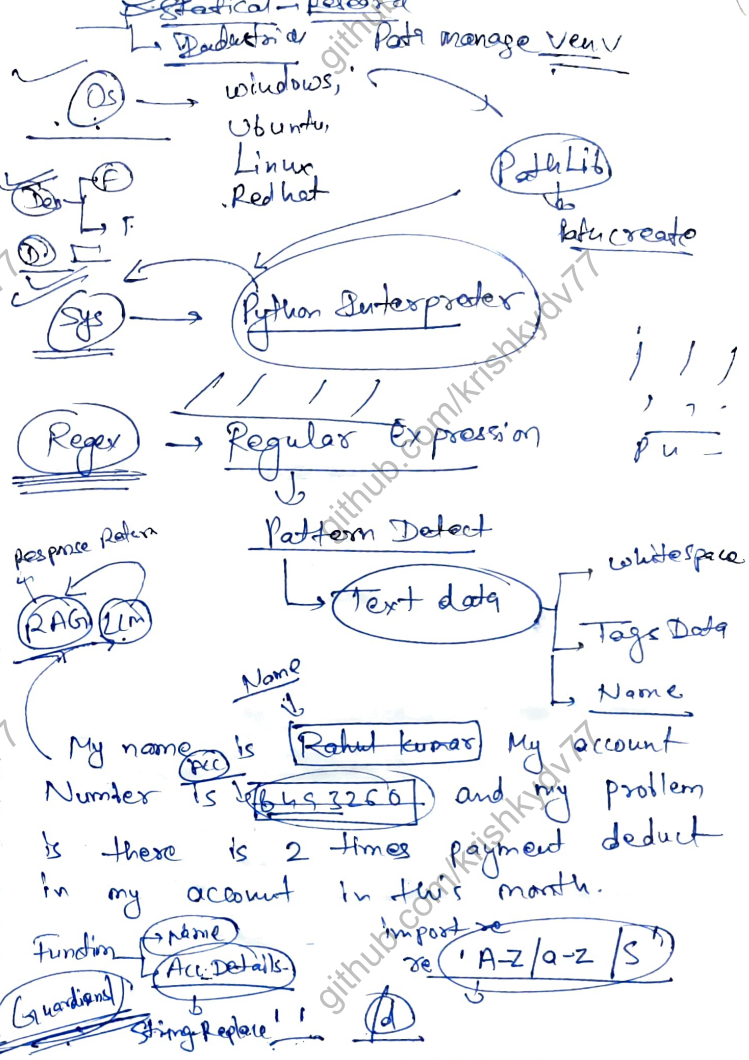
→ Normalization

→ Standardization

to bring all features to the same scale.

Note

KNN is a supervised, non-parametric, instance-based learning algorithm that classifies a new data point based on the majority class of its K-nearest neighbors. It is called a lazy learner because it store training data and performs computation only during prediction.



K-Means clustering

K-Means is an unsupervised ML algorithm used to group similar data point into k clusters. It tries to place similar observation in the same cluster and separate dissimilar observation into different clusters.

How K-means works

1. Choose the Number of clusters (K)
2. Initialize Centroids:-
   It Randomly select k data point as the initial centroids.
3. Assign data points :-
   Calculate the distance b/w each data point and every centroid. Assing each data point to the nearest centroid.
4. update centroids:- Now it will compute the new center of each cluster and ~~cent~~ centroid move forward the middle of their assigned points.

5) Repeat Assignment and Update :-

Now Reassign point based on new ~~centroid~~ centroid and update centroid again and again.

6) Stop when centroid Stop changing :-

once centroid no longer change Significantly, the algorithm Converged and the final clusters are obtained.

Advantage of K-Means

1) Simple and easy to understand
2) Fast and computationally efficient
3) Works well with large datasets
4) Clusters are easy to visualize and interpret

Disadvantages of K-Means :-

1) The value of K must be specified beforehand.
2) Sensitive to centroid initialization, different initial Centroids may produce different results.
3) Sensitive to outliers, which can affect clusters quality.
4) performs poorly on non spherical or overlapping clusters.

Interview Questions :-

1) What is K-Means clustering? why is it Called K-Means.

Ans. K-Means is a unsupervised ML algorithm used to divide data into K-clusters.

It is called K-Means because :-
→ K = number of clusters
→ Means ⇒ The centroid of each cluster is Calculated using the mean (average) of its data point.

2) What is a centroid?

A Centroid is the center point of a cluster. It is calculated by taking the average of all data points in that cluster.

3) What is convergence in K-Means?

⇒ Convergence occurs when the Centroids stop changing Significantly and cluster assignments remain unchanged.

4) What is the Elbow Method? How do You select the optimal Value of K?

github.com/krishkydv77

Ans The Elbow Method helps determine the optimal number of clusters.

→ Run K-Means for different Values of K.
→ Calculate WCSS (Within-cluster Sum of Squares)
→ plot K Vs WCSS
→ Choose the K value where the graph forms an "elbow" shape

This point is Considered the optimal Value of K.

③ Why does K-Mean Sometime give different results.

Ans Because centroids are initialized randomly. Different initial centroid may lead to different final clusters.

④ What is K-Means++?

→ K-Means++ is an improved initialization technique that selects initial centroids more carefully. It helps achieve better clustering and faster Convergence.

Q:7 Seppose one feature range from 1 to 1000 and another from 1 to 10. what problem can occur?

Ans The feature with the larger range will dominate the distance calcution, leading to poor clustering results.

Solution:- Apply feature Scaling

Q:8 what happens if K is too small?

Ans Different groups may be merged into the same cluster, resulting in under-clustering and loss of important patterns.

Q:9 what happens if K is too large?

Data may be split into many small clusters, causing over-clustering and making interpretation difficult.

Q:10 Seppose your dataset contains 50 numerical and 5 categorical columns. Can you directly apply K-means.

github.com/krishydv77

Ans. K-Means works only with ~~numerical~~ numerical data because it uses distance Calculation. Categorical variables must be encoded before applying K-Means.

Q.4 In which situation would you avoid using K-Means.

Ans. Avoid K-Means when:-

1. Data Contains Many outliers
2. Clusters are non-spherical or irregularly Shaped.
3. The optimal value of K is unknown.
4. The dataset Contains mainly categorical features.
5. clusters have very different Sizes or densities.

Note

K-Means is an unsupervised ML algorithm that partitions data into K clusters by assigning each data point to the nearest centroid and repeatedly updating the the centroids until convergence is achived.

PCA (Principal Component Analysis)

1. What is PCA?

⇒ PCA is an unsupervised dimensionality reduction technique that transforms a large set of correlated feature into smaller set of uncorrelated variables called principal components, while preserving as much information as possible.

2. ~~What~~ Why do we use PCA?
   1. Reduce dimensionality of the dataset.
   2. Remove redundant feature
   3. Reduce overfitting in ML Models.
   4. Improve Model performance and training speed.
   5. Help visualize high-dimensional data in 2D or 3D.

Working process of PCA:-

1. Standardize the data

~~Scale~~

Scale all features to a similar range so that each feature contributes equally.

mean = 0, Std = 1

① Mean Center the data.

Calculate the mean of each feature and subtract it from every observation.

② ~~Select~~ Compute the Covariance Matrix

The Covariance matrix measure how features vary with respect to each other.

It helps us understand the relationship b/w features.

③ Calculate Eigen Values and Eigen Vectors.

→ Eigen Values determine how much variance exists in those direction.

→ Eigen vectors determine the direction of Maximum variance.

④ Sort Eigen Values in Descending order

→ Arrange eigen values from higest to lowest.

Higher eigen values indicate more important components because they contain more information.

⑤ form principal Components.

Eigen Vector with the largest eigen value → PC1

Eigen Vector with the second largest eigen values → PC2

Continue similarly for the remaining components.

⑥ Select the top principal components

→ Choose the components that preserve most of the variance.

⑦ Transform the original data.

Project the original data onto the selected principal components.

The result is a new dataset with fewer dimensions but most of the important information preserved.

github.com/krishkydv77

Note:-
PCA is an unsupervised dimensionality reduction technique that converts correlated features into a smaller number of uncorrelated principal components while retaining variance from the original data.

PCA Interview Question

① What is PCA? why do we use PCA?

Ans.
PCA is an unsupervised dimensionality reduction technique that converts correlated feature into a smaller number of uncorrelated feature called principal components

Why use PCA :-
- → Reduce dimensionality
- → Remove redundant feature
- → Reduce overfitting
- → Improve training speed
- → Visualize high-dimensional data

② What is a principal component?
A principal component is a new feature created by PCA.
- → PC1 captures the maximum variance.
- → PC2 captures the second highest variance.
- → Principal components are uncorrelated with each other

github.com/krishkydv77

③ Why do we Standardize data before PCA?

PCA is used on Variance.
If feature have different scales, feature with larger values will dominate the PCA result.
Standardization makes all feature have:-
mean = 0
Standard deviation = 1
So every feature contributes equally.

④ Suppose you have 100 features and after PCA you get 10 components what happened to the ~~rest~~ other 90 features.

⇒ The 90 features are not deleted individually.
Their information is combined into the 10 principal components.
PCA keeps most of the important information while reducing dimensions.

⑤ Can PCA convert 50 features into 100 features.

No.
PCA is mainly used to reduce dimensions, not increase them.
The number of principal components cannot be greater than the number of original features.

⑥ What happens if you choose too few principal components?

⇒ Important information may be lost.
As a result:
Model accuracy may decrease.
Underfitting can occur.

⑦ What happens if you choose too many components?

You keep ~~unnecessary~~ unnecessary information and noise.
As a result.
→ Less dimensionality reduction.
→ Higher computation cost
→ Possible overfitting.

github.com/krishkydv77

⑧ A dataset has 100 feature, After applying PCA, accuracy decreases. Why?

Ans. Possible reasons:-

→ Important information was lost.
→ Too few components were selected
→ PCA removed useful feature
→ PCA is not always beneficial for every dataset

⑨ How do you decide the number of principal components?

Use the Explained Variance Ratio

~~Usually~~ usually, select enough components to retain, 90%, 95%, 99% of the total variance.

KNN (K-Nearest-Neighbors)

① What is KNN? Is KNN a parametric or non-parametric algorithm.

⇒ KNN is a supervised learning algorithm used for classification and regression.

KNN is a non-parametric algorithm because it does not assume any fixed data distribution.

② What does K in KNN represent?

K represent the number of nearest neighbors used to make a prediction.

Ex. K=5 → use the 5 nearest data points.

③ How does KNN work?

① choose K
② Calculate distance between the new point and all training points
③ find K nearest neighbors
④ for classification, use Majority Voting
⑤ for regression, take the average value.

github.com/krishkydv77

④ Which distance metrics can KNN use?

Ans. Comon distance metrics
- ① Euclidean Distance
- ② Manhattan
- ③ Minkowski

⑤ Why is feature Scaling important in KNN.

Ans. KNN is distance based
features with larger values can dominate the distance Calculation.
Scaling ensures all features contribute equally

⑥ What happens if K=1?

The prediction depends on only one nearest neighbor.

Advantage → Very flexible
Disadvantage → Sensitive to noise, Can overfit

⑦ What happens if K is very large?

Ans. The Model becomes too Simple.

As a result:
- → Underfitting may occur
- → Accuracy may decease.

⑧ Does KNN have a training phase?

Ans No.
KNN is a ~~lazy~~ lazy learning algorithm.
It Simply stores the training data and performs Calculation during prediction.

⑨ Your KNN Model performs well on training data but poorly on testing data. Why?

Ans. This is usually overfitting.
Possible reasons:-
- → K is too Small
- → Noise in the data
- → Insufficient training data.

github.com/krishnydv77

(10) If your KNN Model gives ~~low~~ low accuracy, what can you do?

Ans.

→ Try different K Values.
→ Scale the feature.
→ Remove noise and outliers
→ Perform feature selection.
→ Use a different distance metric
→ Increase training data.

Apriori Algorithm

Apriori is an Association Rule Mining algorithm that belongs to unsupervised ML.

It main objective is to discover frequent itemsets and generate association rules from from transaction data.

Ex.

Transaction	Item Purchased
T_1	Milk, Bread
T_2	Milk, Bread, Butter
T_3	Milk, Butter
T_4	Bread, Butter
T_5	Milk, Bread, Butter

Count individual Items:-
Milk = 4
Butter = 4
Bread = 4

Support :- Support measures how frequently an itemset appears in the entire dataset

$$\text{Support}(A \to B) = \frac{\text{Transaction Containing Both A and B}}{\text{Total transaction.}}$$

Milk and Bread appear together = 3

$$\text{Support}(\text{Milk} \rightarrow \text{Bread}) = \frac{3}{5} = 0.60 \Rightarrow 60\%$$

⇒ 60% of all transaction Contain both Milk and Bread.

Confidence :-

Confidence measures how likely a customer is to buy item B after buying item A.

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cap B)}{\text{Support}(A)}$$

Milk appear in 4 transaction.
Milk and Bread together appear in 3 transaction.

$$\text{Confidence} = \frac{3}{4} = 0.75, \quad 75\%$$

⇒ Among customer who purchased Milk, 75% also purchased Bread.

Lift :-

Lift measures how strong the association is between two item compared to random chance.

It tells us whether the relationship is meaningful or just a coincidence.

$$\text{lift}(A \rightarrow B) = \frac{\text{Confidence}(A \rightarrow B)}{\text{Support}(B)}$$

Ex.

Confidence = 0.75
Support (Bread) = $\frac{4}{5} = 0.80$

therefore $\text{lift} = \frac{0.75}{0.85} = 0.9375 \approx 0.94$

Interpredation of lift :-

Lift = 1, No association (Item are purchased independently)
lift > 1 ⇒ positive association
lift < 1 ⇒ Negative association

Ex. lift = 0.94 (< 1)
No strong positive relationship between Milk and Bread.

Advantage of Apriori

1. Easy to understand and implement
2. finds hidden relationship in data
3. useful for recommendation system.
4. Helps in product placement
5. Improves cross-selling strategies.

Disadvantages of Apriori :-

1. High memory usage
2. Generates many candidate itemsets
3. Slow on large datasets.

Note

Apriori is an unsupervised association rule mining algorithm used to find frequent itemsets and generate association rules from transaction data. It work using three important metrics: support, confidence, lift. It is widely used in market basket analysis, recommendation system, cross-selling and product placement.

Interview question

1. What is Apriori algorithm? what is its main objective?

Ans. Apriori is an unsupervised ML algorithm used for association rule mining.

Its main objective is to find frequent itemsets and discover relationship between items that are often purchased or occure together.

2. What is a transaction?

Ans A transaction is one completed purchase or one record in the dataset.

3. What is an item and itemset.

⇒ An item is a single product or object in a transaction.

⇒ An itemset is a group of one or more items that appear together in a transaction.

4. What is a frequent itemset?

A frequent itemset is an itemset that appears many time in the dataset and meets the minimum support value.

5. What happen if we increase and decrease the Minimum support?

Ans.

Increase the Minimum support	Decrease the Minimum support
⇒ fewer itemsets are considered frequent	→ more itemsets becomes frequent.
→ The algorithm runs faster	→ more rules are generated.
→ Some useful patterns may be Missed	→ The algorithm becomes slower → Some unimportant patterns may also appear.

⑥ Is high Confidence always good?

No, High Confidence does not always mean a strong relationship.

Sometimes an item is popular and bought by almost everyone, which increases Confidence even if there is no real association.

that is why we also use lift.

⑦ Why is lift considered better than Confidence?

⇒ Lift Considers how Common the second item is.

Confidence only tells us the probability of buying one item after another.

lift tells us whether the relationship is real or just happening by chance.

So, lift given a better measure of association.

⑧ Can duplicate transaction affect Apriori

⇒ Yes, Duplicate transaction increase the support Count.

This may make some itemsets appear more frequent than they actually are.

So, duplicate transaction can affect the final result.

⑩ A Bank wants to identify services that customers usually subscribe to together. Can Apriori help?

Ans. Yes, Apriori can find ~~transaction~~ services that customer frequently subscribe to together.

The bank can use this information for:

→ Cross-Selling

→ Personalized offers

→ Better Marketing

⑫ A hospital want do discover diseases that frequently occur together. Can Apriori solve this problem?

Yes, ~~Apriori~~ Apriori can find diseases that often occur together in patients.

This helps doctors to:

→ Understand diseases patterns.

→ Improve diagnosis

→ Plan better treatment.

github.com/krishkydv77

Boosting

Boosting is an ensemble learning Method which combines multiple weak or base learners to create a stronger predictive model. The main idea behind boosting is to sequentially train a series of models where each subsequent model focusses on the ~~instances~~ instances that were misclassified by previous models. The iterative process allows the ensemble to learn from its mistakes and improve its overall performance.

Boosting process :-

① Initialization :-> Each instance in the training set is assigned an equal weight.

② Iterative training :- A base learner is trained on the weighted training set and its predictions are evaluated.

③ Weight update :- The weights of misclassified instances are increased. ~~So that the next~~ ~~Model focuses more on these difficult cases.~~

④ while correctly classified instances receive lower weights.

④ Ensemble Creation :- The trained base learner is added to the ensemble with a weight that depends on its performance.

Iteration Termination :- The process continues for predefined no. of iterations or until a performance threshold is reached.

final prediction :- The ensemble combines the prediction of all base learners typically using a weighted voting scheme.

github.com/krishkyd77

Types of Boosting:-

① **Adaptive Boosting:-** It assigns higher weights to misclassified instances and focus on those instances during subsequent iterations. It sequentially trains a series of weak learners and combine a result to form an ensemble.

This model is primarily used for binary classification problems.

② **Gradient Boosting:-**

Gradient Boosting builds an ensemble of weak learners in a stagewise manner. Each subsequent model is trained to correct the mistake make by previous model by fitting negative gradient of loss function.

G.B can handle both classification and Reg. task and is often used with decission tree as base learners.

Type of Gradient Boost:-

① **XG Boost (Extreme Gradient Boosting):-**

XG Boost is an optimised implementation of G.B. that offers several enhancements like including parallel processing, regularization techniques and handling missing values. It uses a combination of tree-based model and linear models for boosting which allows it to capture both linear & non-linear relationships in the data efficiently.

② **Light GBM :-**

Light GBM focus on achieving faster training speed and lower memory usage. It uses a novel tree growth algorithm called Gradient Based one-side Sampling to select the most informative instances for building decision trees.

③ **Cat Boost:-** Cat Boost is designed to handle categorical feature directly without the need for extensive data preprocessing.

It incorporates an innovative method to handle categorical variables, which includes applying a combination of ordered boosting, random permutation, and symmetric trees.

Boosting Model Comparison :-

Dataset Size	Best choice
< 5000 rows	AdaBoost / Gradient Boosting
5000 to 100000 rows	XGBoost
> 100000 rows	Light GBM
Many categorical Columns	CatBoost

Industry preference

Industry	Most Model
Banking	XGBoost
Insurance	XGBoost
Healthcare	XGBoost / CatBoost
E-Commerce	CatBoost / Light GBM
Recommendation System	Light GBM
Customer churn prediction	CatBoost / XGBoost

Model	When should you avoid it?
① AdaBoost	Noisy datasets: highly sensitive to outliers and noise.
② Gradient Boosting	Many categorical features: Training can be very slow due to sequential processing
③ XGBoost	When dataset contains many categorical features and you do want to perform manual encoding.
④ Light GBM	Very small dataset as it may overfit
⑤ CatBoost	Purely numerical datasets where its automated categorical handling provide little added benefits.

Interview Question:

Q1. What is Ensemble learning?

Ans Ensemble learning is ML technique where we Combine multiple models to make one better final prediction.

Q:2 Why do we need Ensemble learning?

Ans. We use ensemble learning to improve model accuracy stability. It can reduce errors like overfitting and underfitting. A group of models usually performs better than one single model.

Q:3 What is the difference between Bagging and Boosting?

Bagging	Boosting
Models are trained independently.	Models are trained one after another.
Each model gets a random Sample of data	Each new model focuses more on previous mistakes.
Mainly reduce variance	Mainly reduce bias.
Helps reduce overfitting	Helps improve weak models.
Ex. Random forest	Eg. AdaBoost, XGBoost.

Q4 What is Bootstrap Sampling?

Ans Bootstrap Sampling means creating many random Samples from the original training data with replacement.

"With replacement" → the same row can be selected more than one time in one Sample.

Q:5 In ensemble learning, which algorithm reduces variance and bias?

Ans.
- → Bagging reduces variance
- → Boosting reduces bias.
- → Random forest mainly reduces variance.
- → Some advanced boosting algorithm, such as XGBoost, can reduce both bias and variance when tuned properly.

Q:6 Can Bagging use the same training Sample multiple times?

Ans Yes, Bagging uses bootstrap Sampling, so the same training record can appear multiple times in one Sample.

github.com/krishkydv77

Q:7 What is (out-of-Bag Error?)

Ans Out-of-Bag (OOB) error is the error calculated using the data rows that were not selected in a bootstrap sample.

In bagging, around one-third of the data is usually left out from each bootstrap sample. This left-out data is used to test the model. OOB error helps us check model performance without needing a separate validation dataset.

Q:8 Your model is overfitting badly. Which ensemble method would you choose and why?

Ans. I would choose Bagging, especially Random forest.

Bagging trains many models on different random samples and combines their predictions. This reduces variance and helps control overfitting.

Q:9 Your model is underfitting. Which boosting technique would you use?

I would use Gradient Boosting or AdaBoost.

Underfitting means the model is too simple and has high bias. Boosting trains models step by step and focuses on previous mistakes. So it reduces bias and improves learning.

DBSCAN

Q:1 What is DBSCAN? What is the main objective of DBSCAN.

Ans. DBSCAN stand for Density-Based Spatial clustering of Applications with Noise.

It is a clustering algorithm that groups data points which are close together and have density. It can also identify unusual points as noise or outliers.

The main objective of DBSCAN is to find clusters of different shapes and separate noise points without telling the algorithm the number of cluster in advance.

Q:2 What are Epsilon and MinPts? What happens if epsilon is increased or decreased?

Ans.

Epsilon (eps):- It is the Maximum distance between two points to consider them as neighbors.

MinPts:- It is the Minimum number of point required inside the epsilon distance to create a dense region or cluster.

If epsilon is increased:-

→ More points become neighbors.
→ Small clusters may merge into one large cluster.
→ Fewer points are marked as noise.

If epsilon is decreased:

→ Fewer points between neighbors.
→ One cluster may break into smaller clusters.
→ More points may be marked as noise.

Q:3 How does DBSCAN work?

Ans DBSCAN works in these steps:

1. It selects one data point.
2. It finds all neighboring points within the epsilon distance.
3. If the point has at least MinPts neighbors, It becomes a core point.
4. DBSCAN create a cluster around that core point.
5. It keeps adding connected nearby points to the same cluster.
6. Points near a core point but having fewer than MinPts neighbors are called border points.
7. Points that do not belong to any cluster are called noise points or outliers.

github.com/krishnaiyadv77

Q.4 What is the difference between DBSCAN and K-Means?

DBSCAN	K-Means
It is a density-based clustering algorithm.	It is a Centroid-based clustering algorithm.
It does not require the number of clusters in advance.	It requires the values of K before training.
It can find clusters of different shapes.	It works best with round or spherical clusters.
It can identify noise and outliers.	It assigns every points to a cluster, even outliers.
It uses epsilon and MinPts parameters.	It uses K and Centroids.
It can struggle when clusters very different densities.	It can struggle with outliers and non-round clusters.

Optimization Technique

⇒ Loss function Minimize
↳ (y_act − y_pred)

⇒ for regression, a common loss function is Mean-squared Error (MSE):-

$$L = \frac{1}{n}\sum_{j=1}^{n}(y_i - \hat{y}_i)^2$$

y_i = actual value
$\hat{y}_i$ = predicted value
n = total number of data rows
L = Loss.

Type of Gradient Descent :-

① Batch Gradient Descent:-

Batch Gradient Descent uses the entire dataset to calculate the loss and update the Model parameters.

Ex. if 100 rows → It use all 100 row before making one update.

Advantage :- Stable update
Disadvantage => Slow when the dataset is very large.

② Stochastic Gradient Descent (SGD):-

Stochastic Gradient Descent uses one data row at a time.

Ex: 100 row → It calculates loss using one row, update the model, then moves to the next row.

Advantage:- faster updates and useful for very large datasets

Disadvantage:- update can be noisy because one row may not represent the full data.

③ Mini-Batch Gradient Descent:-

Mini-Batch Gradient Descent uses a small group of row at a time.

Ex: 100 row, we can create 4 batches:

Batch (A, B, C, D) = rows (25, 25, 25, 25)

The Model update after each batch.

It is commonly used because it is faster than Batch GD and more stable than SGD.

Gradient Descent for linear Regression:

The equation of simple straight line is:

$\hat{y} = mx + b$

$\hat{y}$ = predicted value
x = input feature
m = slop or weight
b = intercept or bias

for one data point, the squared error loss

$L = (y - \hat{y})^2$

Substitute $\hat{y} = mx + b$

$L = (y - (mx + b))^2$

$L = (y - mx - b)^2$ → ~~इसको~~ Minimum करना है।

Derivative with Respect to intercept b. (Bias के लिए)

$$\frac{\partial L}{\partial b} = \frac{\partial}{\partial b}(y - mx - b)^2$$

$$= 2(y - mx - b)(-1)$$

$$= -2(y - mx - b) \quad \text{---(1)}$$

→ यह बताता है bias कितना है और direction में change चाहता है।

Derivative w.r.t → m

$$\frac{\partial L}{\partial m} = \frac{\partial}{\partial m}(y - mx - b)^2$$

$$\frac{\partial L}{\partial m} = 2(y - mx - b)(-x)$$

$$= -2x(y - mx - b) \quad \text{—②}$$

यह बताता है कि weight में कितना और किस direction में change करना है।

Updating weight and Bias :-

New Value = old Value − α × Gradient

Gradient Descent update the value of m and b using the learning rate.

$$b_{new} = b_{old} - \alpha \frac{\partial L}{\partial b}$$

$$m_{new} = m_{old} - \alpha \frac{\partial L}{\partial m}$$

α ⇒ learning rate.

↳ decides how big each update step will be.

⇒ Very small α ⇒ learning becomes slow

Very large α ⇒ model may skip the minimum loss point

Proper α ⇒ loss gradually decrease

After update m and b, the new regression line becomes:-

$$\hat{y} = m_{new}x + b_{new}$$

This process repeats many time until the loss becomes minimum.

$$\frac{\partial L}{\partial b} = -\frac{2}{n}\Sigma(y - \hat{y})$$

$$\frac{\partial L}{\partial m} = -\frac{2}{n}\Sigma x(y - \hat{y})$$

Gradient descent :-

→ optimization algorithm

→ It helps the model find better value of weight and bias so that loss become minimum. (which direction.

⇒

Ex. Let.

$x = 2, \quad y = 50$

Starting Value $m = 5, \; b = 30, \; \alpha = 0.01$

$$\hat{y} = mx + b$$

$$\hat{y} = 10 + 30 = 40$$

Error = $y - \hat{y} = 50 - 40 = 10$

Loss :-

$$L = (50 - 40)^2 = 100$$

Gradients

$$\frac{\partial L}{\partial b} = -2(50 - (10 + 30)) = -20$$

$$\frac{\partial L}{\partial m} = -2 \times 2(10) \Rightarrow -40$$

Weight / Bias update

$b_{new} = 5 - 0.01(-90) \Rightarrow 30.2$

$m_{new} = 5 - 0.01(-49) \Rightarrow 5.4$

New line

$y = 5.4x + 30.2$

lets $x = 2$

$y = 41$

prediction = 40 ~~अब~~ ⇒ 41

actual value = 50, ~~मास~~ पास हो गया Loss ~~कम~~ होगा।

github.com/krishkydv77

① What is mysql.connector?

Ans. Mysql.connector is a ~~Mysql~~ python library used to connect python with mysql database. We can run sql queries and fetch data in python.

② ~~Why~~ which function is used to create a mysql connection? → Mysql.Connector.connect()

④ What details are required to connect Mysql with python?

⇒ Hostname, username, password, database

Code

~~Mysql.c~~

```
Import Mysql.connector

conn = Mysql.Connector.Connect(
    Host = " "
    username = " "
    Password = " "
    database = " " )
```

① What is a cursor in mysql connector?

Ans. A cursor is an object used to execute sql queries and ~~fetch~~ fetch results from the database.

fetchone() ⇒ returns only one row

fetchall() ⇒ returns all row from the query result

Q. Why should we close the connection?

⇒ We close the connection to free database resources and avoid unnecessary open connections.

```
cursor.close()
conn.close()
```

④ load Mysql data → Pandas Dataframe

```
query = "select * from table"
df = pd.read_sql(query, conn)
```

Data cleaning

⇒ Data cleaning is the process of fixing or removing incorrect, missing, duplicate and unwanted data.

Important ⇒ ML Models learn from data. If data is dirty, the model can given wrong prediction.

Check Missing Value ⇒ `df.isnull().sum()`

Remove rows with Missing Value ⇒ `df.dropna(inplace=True)`

fill Missing numeric Value ⇒

```
df["column"].fillna(df["column"].mean(), inplace=True)
```

⑲ How should we use Median instead of Mean?

⇒ use Median when the column has outliear because Median is less affected by very high or very Low Value.

fill Missing Categorical Value.

```
df["column"].fillna(df["column"].mode()[0], inplace=True)
```

⇒ check duplicate ⇒ `df.duplicated().sum()`

remove duplicate row = `df.drop_duplicates(inplace=True)`

feature selection ⇒ choose important existing columns.

feature Extraction ⇒ Create new feature from existing columns.

feature Selection Method →
- filter Method
- wrapper Method
- Embedded Method.

filter Method ⇒ Statistical technique before training the Model.

wrapper Method ⇒ check different combination of feature by training a model many times.

embeded Method ⇒ Selects feature while the model is training.

Correlelation-based feature selection?

⇒ It checks the relationship b/w feature and the target.

Multicollinearity ⇒ when two or more input features are highly related to each other.

problem ⇒ coefficient unstable / difficult to interpret

New

RFE ⇒ (Recursive feature Elimination)

It repeatedly remove the least important feature until the required number of feature remains.

① feature scaling ⇒ Converting feature value into a similar Range.

↳ need ⇒ because some algorithm use distance or Math. Calculation.

Normalization :-

↳ It is feature scaling technique that convert value into a fixed range, usually 0 to 1.

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Standard Scaler

It change data so that:

Mean → 0

Standard deviation become 1

$$X_{new} = \frac{X - mean}{Std\text{-}d.}$$

⇒ Negative, positive, zero

fit_transform()

fit() → learn value from data

transform() → change the data using those learned value

day-2

outlier

outlier is a data value that is very different from most other values in data set.

① detecting outlier using IQR method:-

Q1 = df["Colem"].quantile(0.25)

Q2 = df["Colem"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = $Q_1 - 1.5 * IQR$

Upper_limit = $Q_3 + 1.5 * IQR$

outliers = df[(df["Column"] < lower_limit) | (df["Column"] > upper_limit)]

① Trimming Method :-

Trimming means removing row that contain outlier values.

When use :-

① The value is clearly wrong
② It is a data-entry error
③ remove row
④ outlier is not important for the business problem.

~~df = df.copy()~~

trimming_method = df[(df["Column"] >= lower_limit) & (df["Column"] <= upper_limit)]

② Capping :-

Capping means we do not remove the row. Instead we replace the outlier the value with the lower or upper limit.

use: → data row is important
→ do not lose record
→ data set is small.

Capping_Method = df.copy()

Capping_Method["Column"] = df.Capping_M["Column"].clip(lower = lower_limit, upper = upper_limit)

print(capp_M)

Point	Trimming	Capping
Meaning	Removes outlier rows	Replace outlier values.
Data rows	Decrease	Stay the same.
Best when	Value is wrong	Row is useful
Risk.	Important data may be lost	original extrem Value is changed,

Outlier are extreme value that are very different from the majority of data. The can effect model performance. we can handle them using trimming where we remove outlier rows - or capping, where we replace extreme value with lower or upper limit.

Topic: Not

Column Transformer

↳ CT is a tool from scikit-learn

↳ CT that we apply different preprocessing step to different columns in the same dataset.

from sklearn.compose import ColumnTransformer

Preprocessor = ColumnTransformer(transformers = [("num", numerical_transform, column), ("cat", cat_trans, column)])

preprocessor.fit_transform(df)

Missing Value

for from sklearn.impute import SimpleImputer

Algorithm	Use when	Don't use when	Main Advantage	Main limitation
K-mean clustering	Data has clear cluster and similar size	cluster have different shapes or many outliers.	fast and easy	Need to choose K value
DBSCAN	Data has irregular-shaped clusters and noise	Data density is very different.	~~Data density is very different~~ No Need to choose K first.	Slow for large data
PCA	Need to reduce feature	Relationships are non-linear.	Reduces dimensions and Speeds up models	Information loss
Apriori	Need association rules (Market Basket Analysis)	very large number of items.	Easy to understand rules.	Slow with many items.

github.com/krishkydv77